

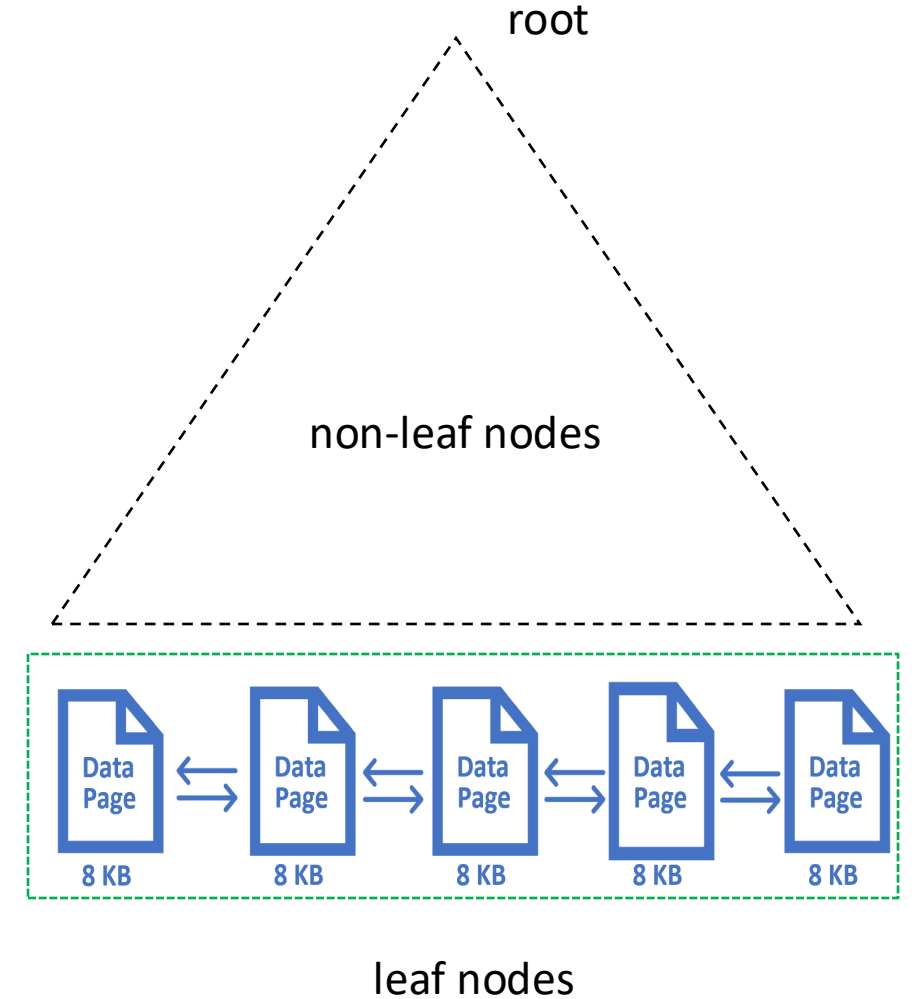
Veri Organizasyonu (289)

Doç. Dr. Ahmet Arif AYDIN

L11 – B+ Ağacı ve Hash Indexleme

Güz Dönemi -2024

- B+ Ağaç yapısında bulunan düğüm çeşitleri hangileridir?
- B+ Ağacındaki düğümlere (sayfa) eklenecek eleman sayısı neye göre belirlenir?
- B+ Ağacının yüksekliği ve depolanacak verilerin sayısı hangi formül ile hesaplanır?
- B+ Ağaç yapısında arama (search) işlemi nasıl başlar ve nasıl devam eder?
- B+ Ağaç yapısında insert (ekleme) işlemi nasıl çalışır?

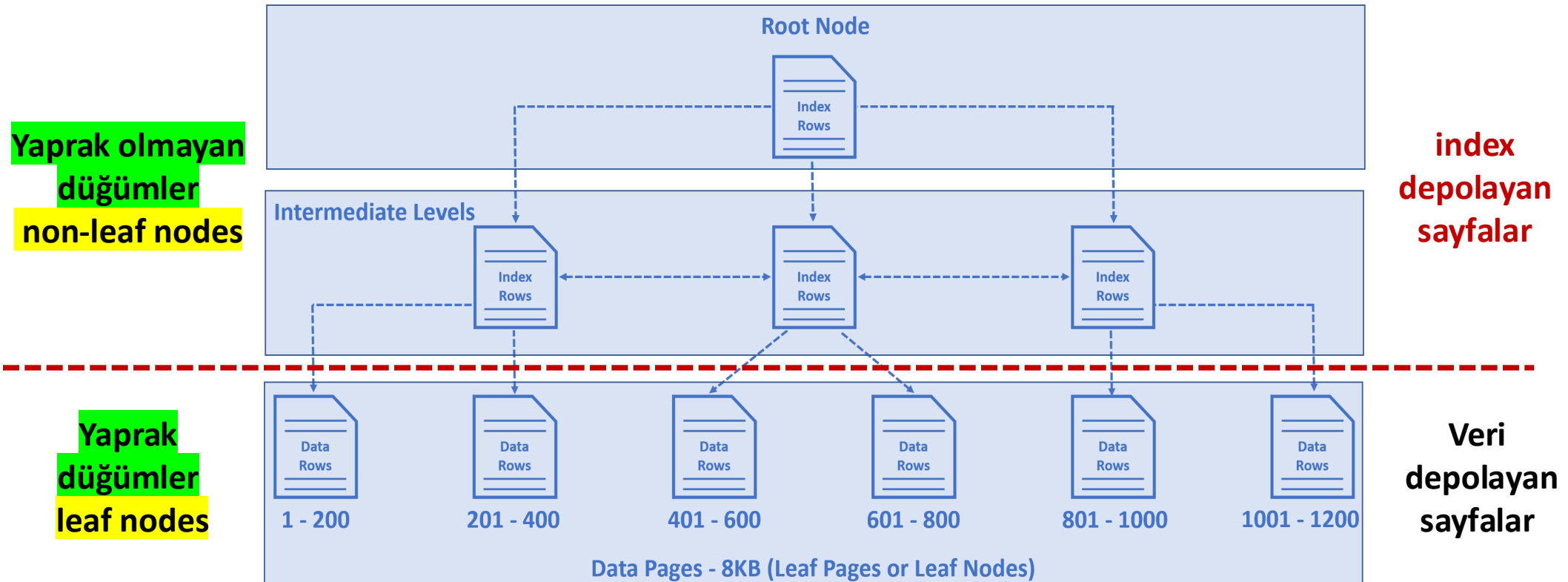


➤ B+ Ağacının özellikleri

Employees Table			
EmployeeId	Name	Email	Department
1	Mark	mark@pragimtech.com	IT
2	John	john@pragimtech.com	HR
3	Sara	sara@pragimtech.com	HR
4	Mary	mary@pragimtech.com	IT
5	Dave	dave@pragimtech.com	IT
...			
1200	Steve	steve@pragimtech.com	HR

İlişkisel Veritabanı Yönetim sistemlerinde (RDBMS)
varsayılan (default) veri depolama mekanizması B+ Ağacıdır

- B+ ağacının **esnek ve dinamik bir yapısı** bulunmaktadır
- B+ ağacındaki her bir düğüm **disk üzerinde bir sayfadır**.



➤ B+ Ağacı: Arama (Search)

1. B+ ağacında arama işlemine **root (kök düğüm)** ile başlanır
2. Düğümün türüne göre a veya b seçeneği çalıştırılır
 - a) Arama yapılan düğüm **yaprak olmayan düğüm (non-leaf node)** ise
 - Anahtar değerine göre düğümde arama yapılır
 - **Aranılan anahtar değerden küçük olan en yüksek değerli anahtardan sonraki işaretleyicinin gösterdiği düğüme gidilir.**
 - Arama yapılan düğüm güncellenir
 - 2.adıma geri dönülür.
 - b) Arama yapılan düğüm bir **yaprak düğüm (leaf node)** ise
 - **aranılan değer bu düğümdedir veya yoktur.**

1- **Eklenmesi gereken kayıt için (düğüm) uygun düğüm aranır**

2-Eğer bulunan yaprak düğüm maximum depolayabileceği değerden az kayıt içeriyorsa kayıt sırası dikkate alınarak ekleme yapılır (işlem tamamlanır)

3- Eklenecek değer için düğümde yer yoksa

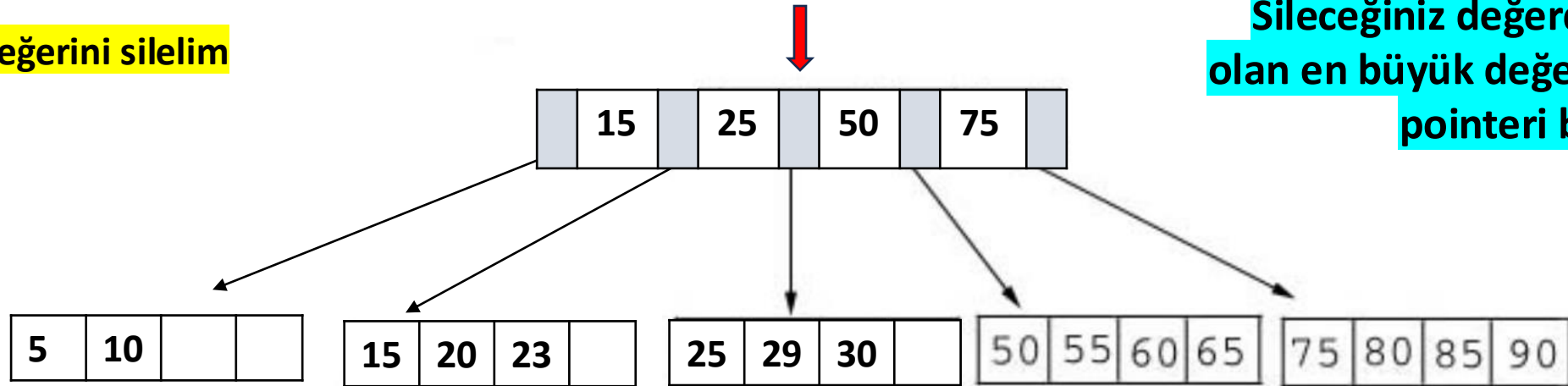
- Düğüm ikiye bölünür, yeni bir yaprak (leaf) oluşturulur
- Elemanların yarısı sırası değişmeden oluşturulan yeni düğüme(yaprağa) eklenir.
- ***Yeni oluşturulan düğümün anahtarı bir üst seviyedeki düğüme eklenir***
- ***Üst seviyedeki düğüm dolu ise ikiye ayrılır ve ortadaki anahtar üst seviyeye alınır.***
- Bölünme gerektirmeyen düğüm oluşuncaya kadar devam eder.

4- **Eğer root (kök) düğümünün bölünmesi gerekiyorsa tek bir degeri ve iki pointeri bulunan yeni bir root oluşturulur.**

1. Silinecek değerin yaprak düğümü anahtar değeri yardımıyla bulunur.
2. Silme işleminin gerçekleştirileceği düğümde ağacın derecesine eşit veya büyük sayıda kayıt varsa silme işlemi gerçekleştirilir.
3. **$d \leq \text{düğümdeki kayıt sayısı} \leq 2d$** şartını sağlamıyorsa düğümler aynı ebeveyne sahip düğümler birleştirilir
4. Birleştirme işlemi gerçekleştirilirken bir üst seviyede (ebeveyn) silinen düğümü gösteren pointer ve anahtarı silinir
5. Bu işlem root (kök) a kadar devam eder

➤ B+ Ağacı: Silme (Delete)

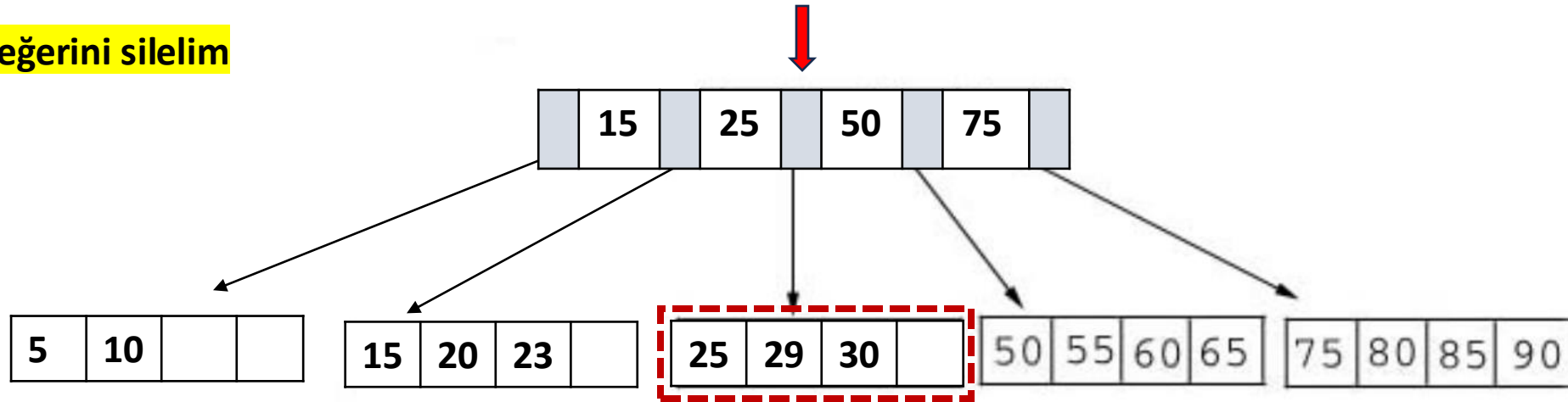
30 değerini silelim



Sileceğiniz değerden küçük olan en büyük değerden sonraki pointeri bul

➤ B+ Ağacı: Silme (Delete)

30 değerini silelim



Pointer'ın gösterdiği
yaprak düğüm mü?

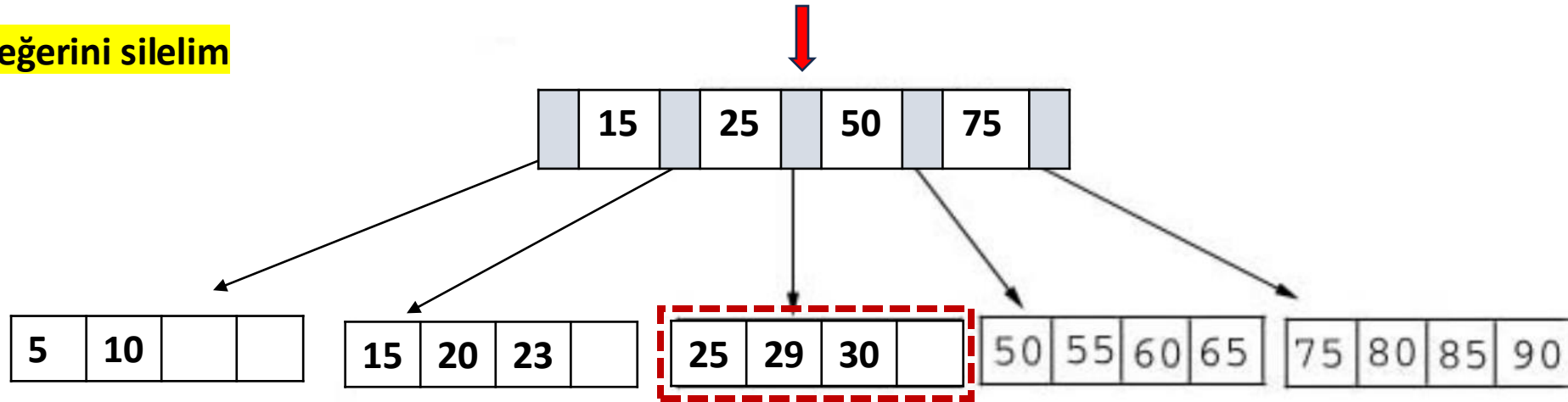
?

30 değeri silindiğinde
 $d \leq$ düğümdeki kayıt sayısı $\leq 2d$
şartı sağlanıyor mu?

?

➤ B+ Ağacı: Silme (Delete)

30 değerini silelim



Pointer'ın gösterdiği
yaprak düğüm mü?

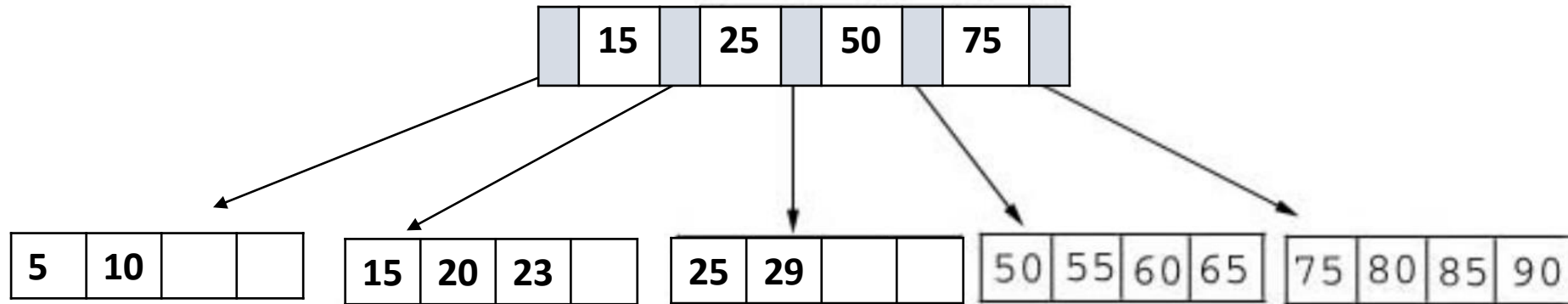


30 değeri silindiğinde
 $d \leq$ düğümdeki kayıt sayısı $\leq 2d$
şartı sağlanıyor mu?



➤ B+ Ağacı: Silme (Delete)

30 değeri silindikten sonra elde edilen yeni ağaç



30 değeri silinir

$d \leq \text{düğümdeki kayıt sayısı} \leq 2d$

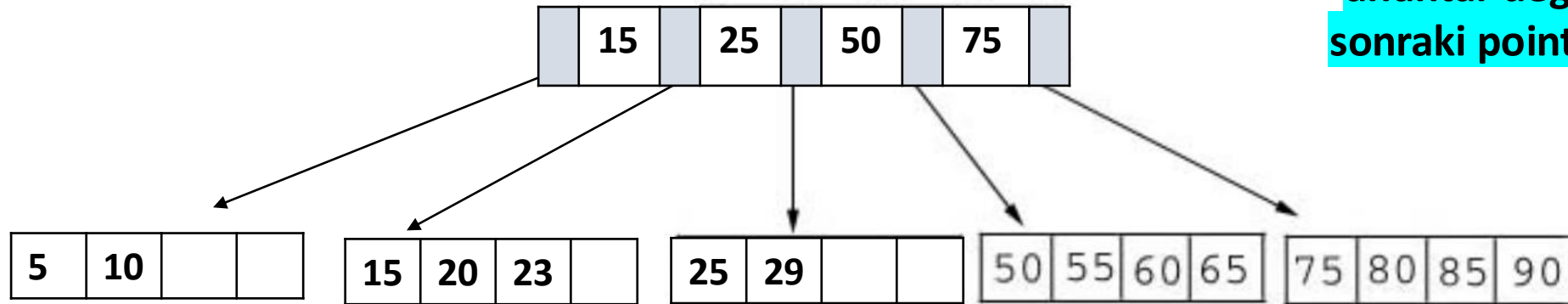
şartı sağlandığından ve düğümünde ponteri bulunduğundan

ağaç yapısında herhangi bir değişiklik gerekmez.

➤ B+ Ağacı: Silme (Delete)

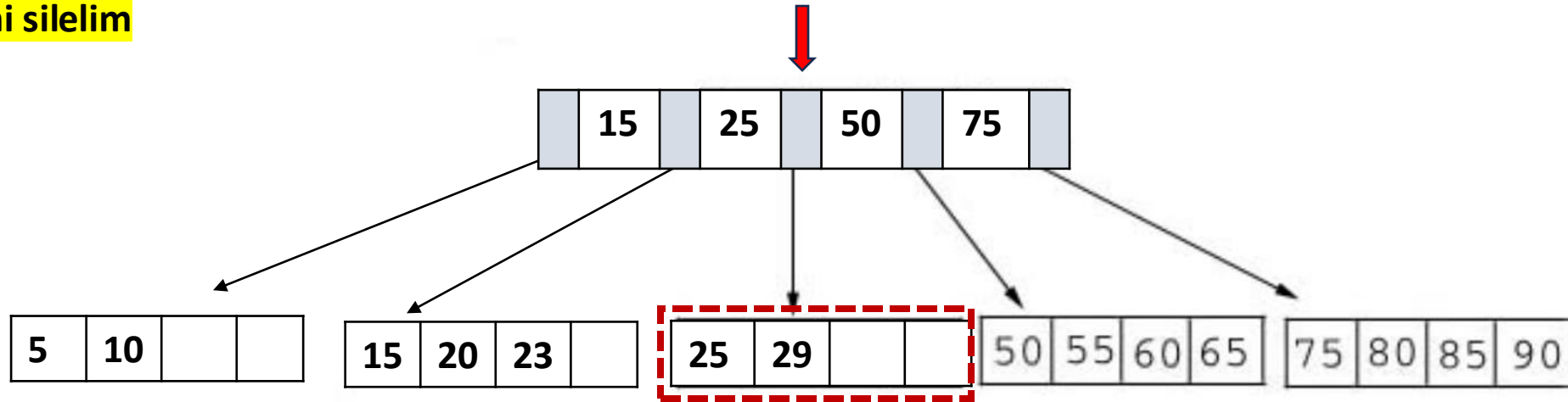
25 değerini silelim

Sileceğiniz değer anahtar ise
anahtar değerden
sonraki pointeri bul



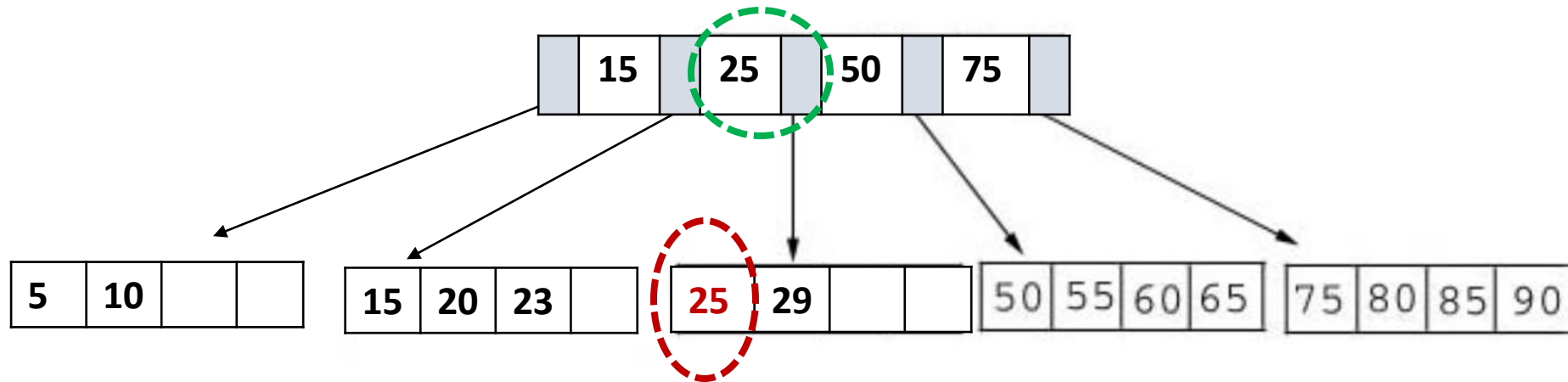
➤ B+ Ağacı: Silme (Delete)

25 değerini silelim



Ponterin gösterdiği düğüm
yaprak düğüm olduğundan
eleman silinebilir.

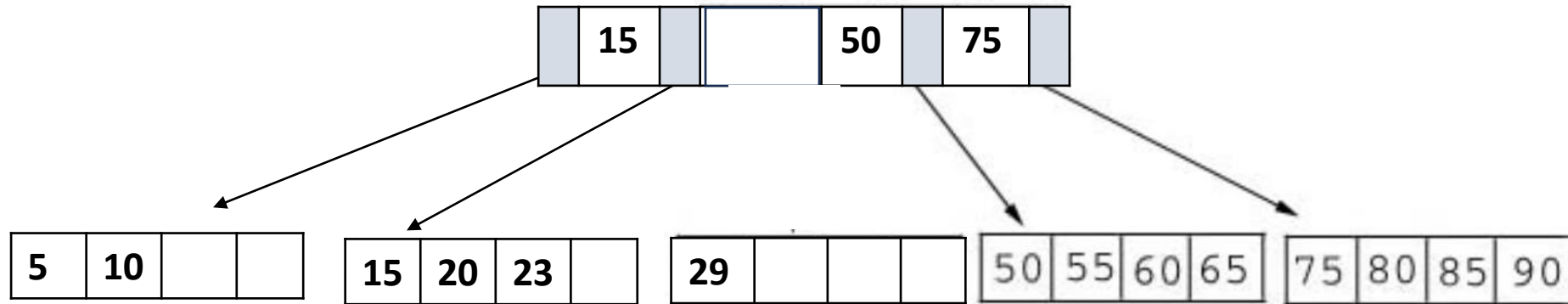
25 değerini silelim



- Yaprak düğümde bulunan **25 değeri silinir**
- 25 değerinin **anahtarı ve pointeri de üst seviyeden silinir**

➤ B+ Ağacı: Silme (Delete)

25 değerini silelim



29 değeri yaprakta tek başına kalmaz

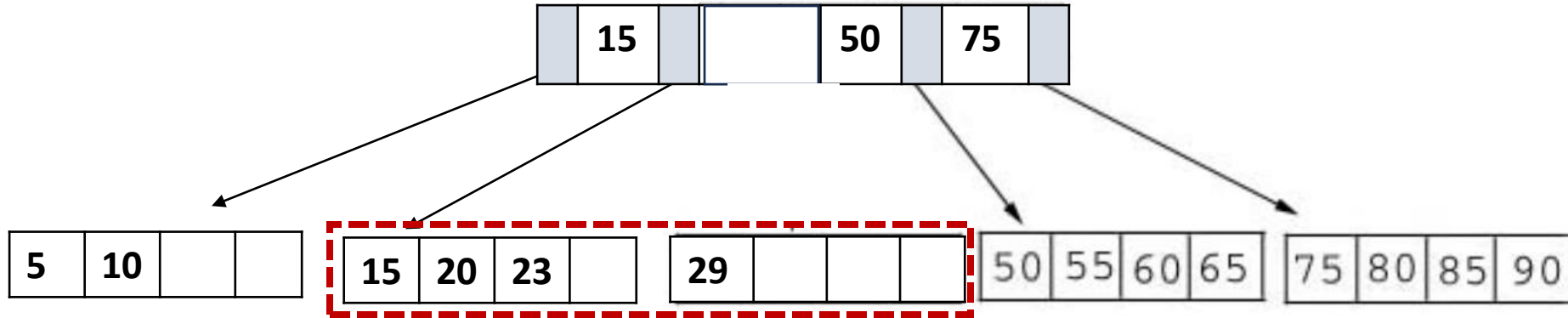
$d \leq \text{düğümdeki kayıt sayısı} \leq 2d$

şartı sağlanmamaktadır.



➤ B+ Ağacı: Silme (Delete)

25 değerini silelim



29 değeri yaprakta tek başına kalmaz

$d \leq \text{düğümdeki kayıt sayısı} \leq 2d$

şartı sağlanmamaktadır.

Silme işleminde

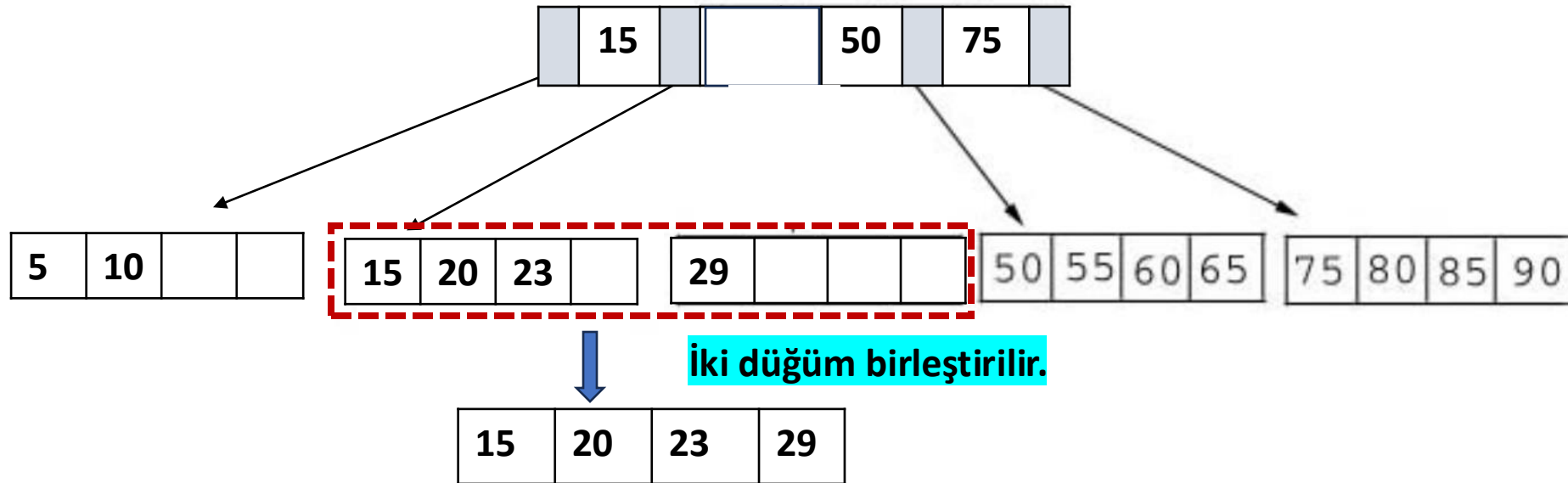
doluluk şartını sağlamayan düğüm

aynı ebeveyne sahip

uygun olan bir düğüm ile birleştirilir.

➤ B+ Ağacı: Silme (Delete)

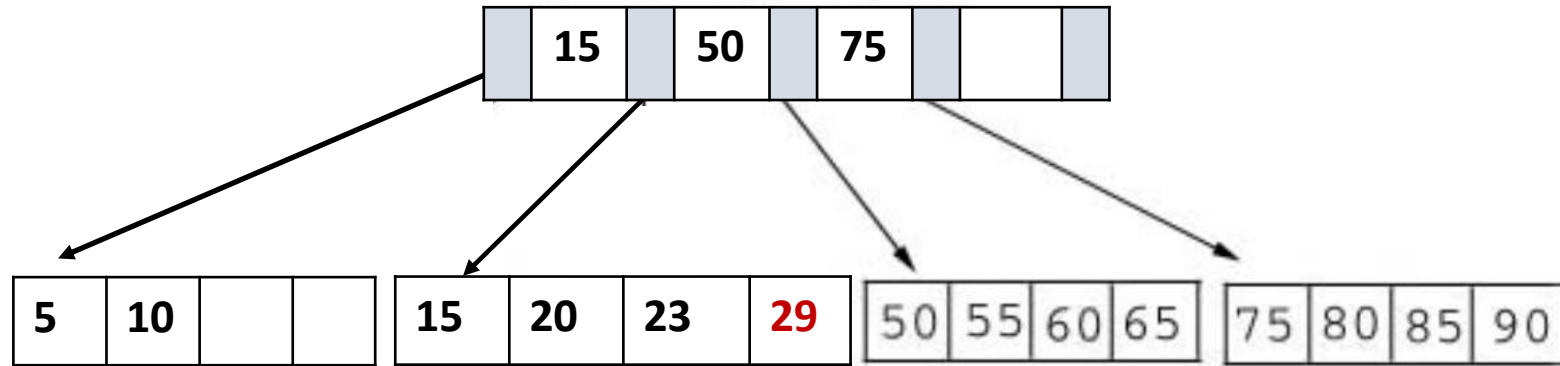
25 değerini silelim



29 değeri yaprakta tek başına kalmaz
 $d \leq \text{düğümdeki kayıt sayısı} \leq 2d$
şartı sağlanmamaktadır.

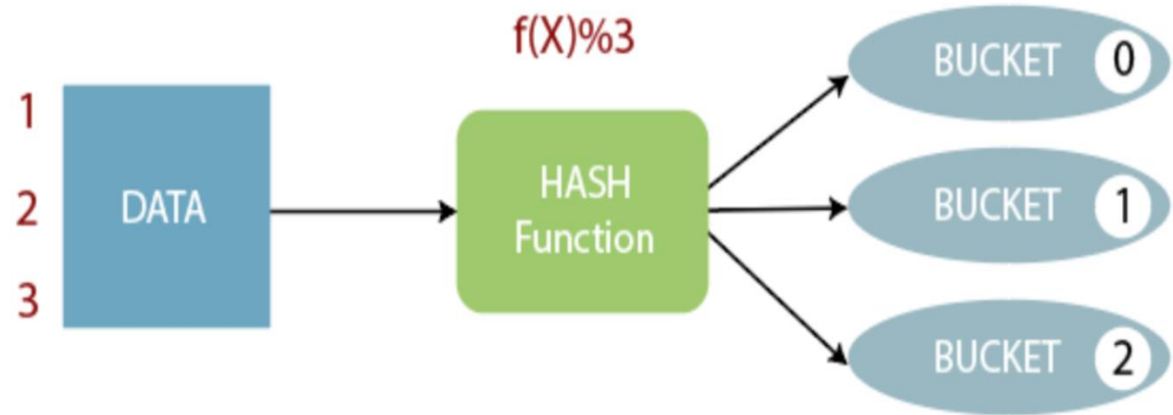
Silme işleminde
doluluk şartını sağlamayan düğüm
aynı ebeveyne sahip
uygun olan bir düğüm ile birleştirilir.

➤ B+ Ağacı: Silme (Delete)

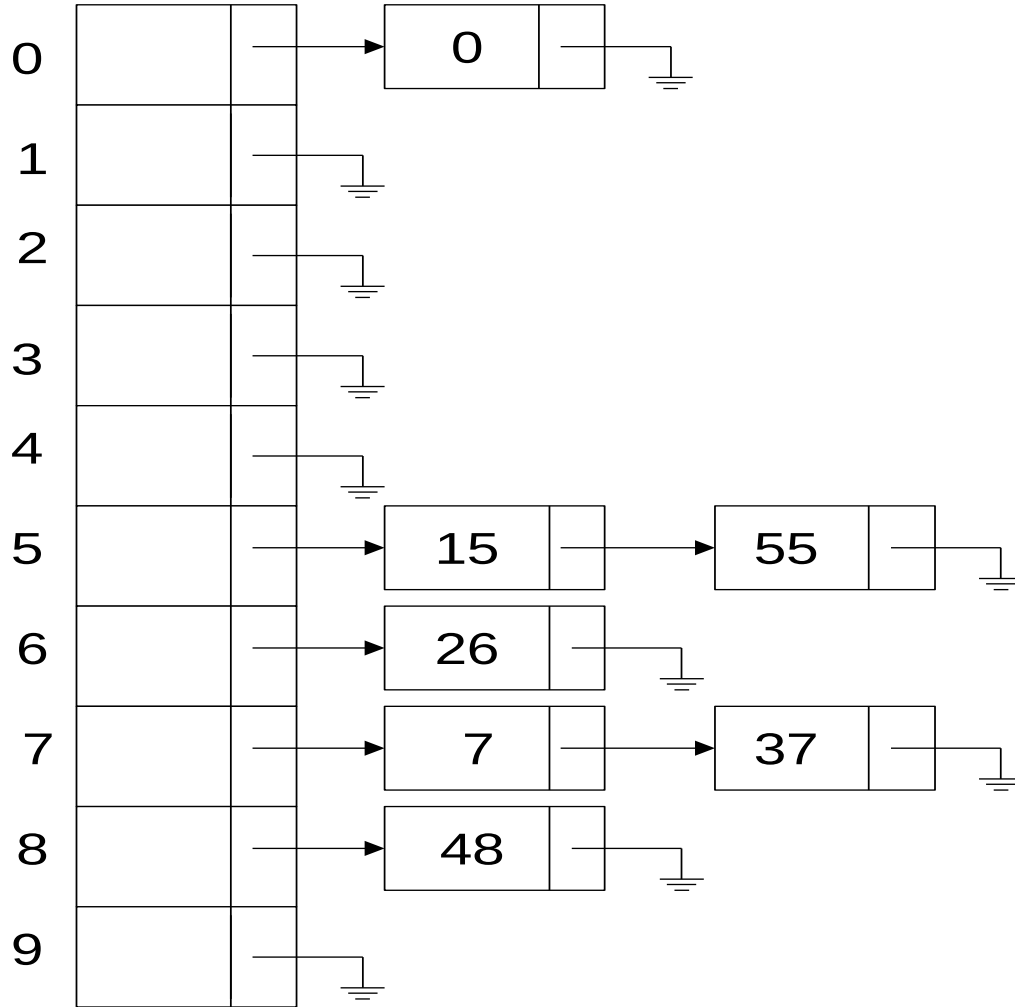


25 değerini silindikten sonra elde edilen yeni ağaç

- VTYS konseptinde Hashing, **bir veritabanında ki bir veri kaydını hızlı bir şekilde bulmak için kullanılan bir tekniktir.**
- Özellikle performans ve veri alma süresinin en aza amacıyla **hashing** tekniği kullanılır



➤ Hash Index



$$h(x) = x \bmod 10$$

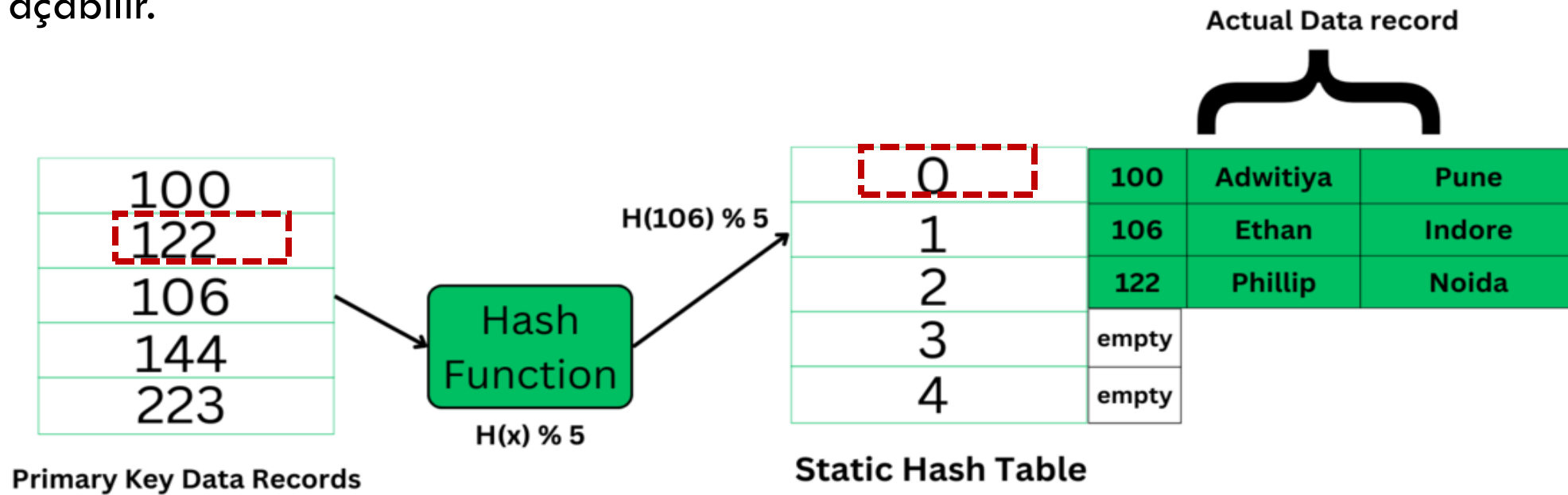
hash (çarpı) fonksiyonuna göre

$K = \{0, 7, 15, 26, 37, 48, 55\}$ elemanlarının hash tablosuna eklenmiş hali

- Hash fonksiyonunun sonucuna göre verinin gruplara ayrılma işlemi gerçekleşir.
- Yeni değerler eklenirken hash fonksiyonunun sonucuna göre bucket belirlenir ve değer eklenir.
- Her bir alt grup **bucket** (kova) olarak isimlendirilir.

➤ Hash Index: Static Hashing

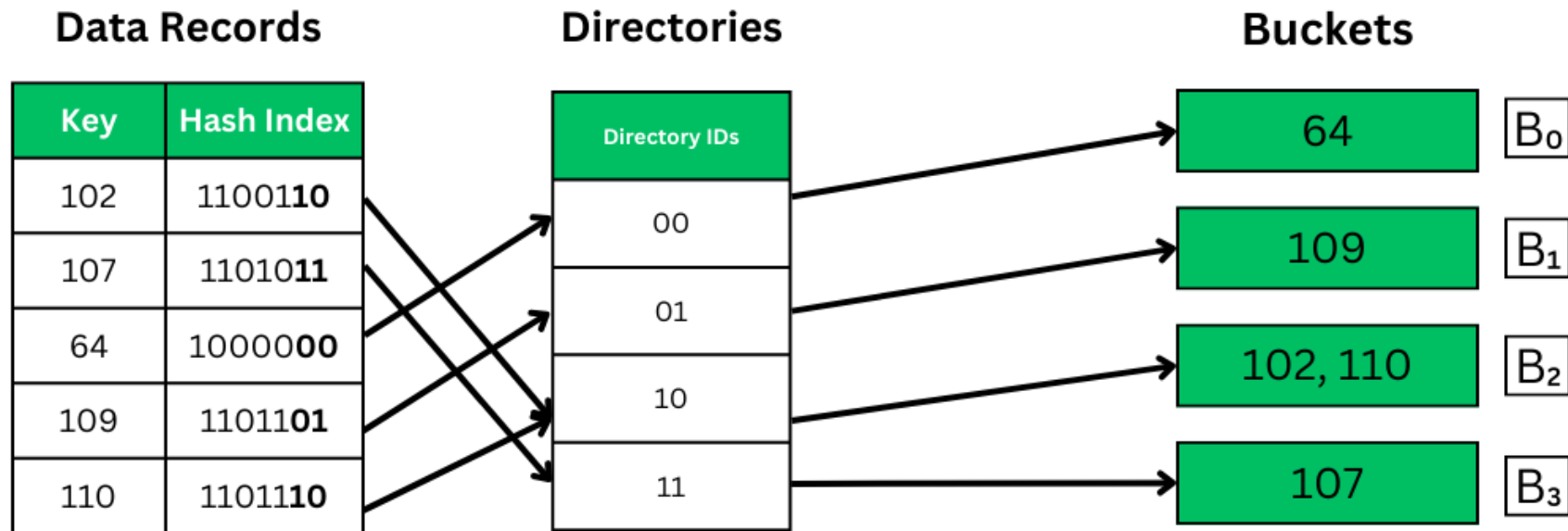
- Statik hashing PK anahtarı hash fonksiyonuna gönderilir.
- Statik hashing de hash fonksiyonun sabit sayıda kova adresi üretir. $H(x) \% 5$
- Sürekli değişen veri setleri için uygun değildir. Özellikle anahtar sayısı öngörülemediğinde veya zaman içinde değiştiğinde taşma (overflow) ve anahtarların kötü dağılımı gibi sorunlara yol açabilir.



<https://www.geeksforgeeks.org/hashing-in-dbms/>

➤ Hash Index: Dynamic Hashing

- Dinamik hashing (**extensible hashing**) sürekli değişen veri setleri için uygundur.
- Dinamik olarak bucket (veri kovaları) ekleme ve çıkarma imkanı sunar.
- Veri kayıtlarının sayısı değiştikçe yada bir değişiklik yapıldığında kovaların boyutu da periyodik olarak büyüyecek ve küçülecektir.



<https://www.geeksforgeeks.org/hashing-in-dbms/>

Dinlediğiniz için Teşekkürler...
İyi çalışmalar ve başarılar dilerim ...
